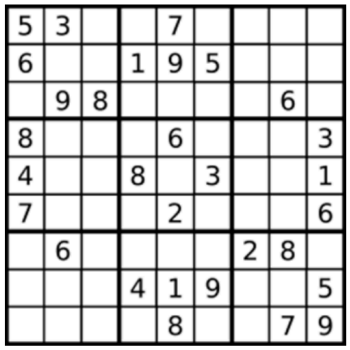
Analysis:

a. Name of two GenAI programs:
   Google Gemini 3 and Claude.ai Sonnet 5.5 Medium
b. How you accessed them (e.g., Google query, CoPilot on my computer)
   Google query and Claude chat box
c. Prompt used. (You must use the same prompt for both GenAIs.)

Assignment:

1. Use two GenAI's to generate C++ code using objects, not functions, to implement the following requirements (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI.):
   - Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
   - A typical puzzle is shown here (image credit: Wikimedia Commons):
     - 
   - Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:
     - Each digit from 1 to 9 must appear exactly once in each row.
     - Each digit from 1 to 9 must appear exactly once in each column.
     - Each digit from 1 to 9 must appear exactly once in each block.

- Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.
- The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.
- It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it's possible to gradually fill in the entire grid.
- That is not the approach we're going to use. Instead, we're going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by "slower to run," we mean it'll finish in a few tenths of a second rather than a few milliseconds.)
- Here's the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists

(all candidates report no solution, which again implies we took a wrong path somewhere earlier).

- This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.
- You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
- Write a **recursive object-oriented** program to find the solution to the five puzzle files.
- The output for each solution, should include:
  - The puzzle file name (e.g., puzzle1.txt)
  - The puzzle from the file printed out.
  - The solution to the puzzle printed out or if no solution is found, "No solution found" printed out.
- The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.
- To help you debug your program, solution1.txt is a solution to puzzle1.txt.

Adequate comments summarizing major blocks of code and comments on every line:

- Provide comments that explain what each line of code is doing.
- You may comment each line of code and/or provide a multi-line comment that explains what a group of lines does.
- Multi-line comments should be detailed enough that it is clear what each line of code is doing.
- Each block of code must indicate whether you authored the code, you obtained it from one of the sources listed in the prolog, or one of your collaborators authored the code, or if it was a combination of all of these.

[part d is at the end of the pdf]

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

1. Correctness: Claude.ai wins this category. Gemini produces the desired output, but without the intended user to terminal relationship; instead of taking the puzzle text files as input, Gemini runs and prints its solutions for all 5 puzzles, which are hard-coded into the main file, without reading the input arguments. Claude executes similarly by printing its solutions to all 5 puzzles, but only if no file arguments are provided.

2. Execution time & Space complexity: Gemini wins this category. The extra 52 lines of code

   // Gemini wins in this section; despite its extra 59 lines of code, it includes less external libraries that cost more space and execution time compared to Claude.ai’s.
3. Maintainability:
   // Claude.ai’s code works for any user input provided, so it automatically wins as a versatile program in terms of maintainability.

f. Select one to use as the basis for this assignment and justify why you picked it.

// I prefer Claude.ai’s code since it works with user input, improving maintainability and making debugging easier with other test files.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

The program’s length is now 126 lines of code, practically a happy medium of

// I pruned the extra libraries Claude.ai’s code used and developed a hybrid version that functions like Gemini’s, but with better correctness and maintainability.

[part 3 is at the end of the pdf]

d. Generated code from both GenAIs. (also available in github repo in GenAI code folder)

Google Gemini code (96 lines, excluding comments):

```
// Source Attribution: Authored by student with assistance from Generative AI
// Description: Main inclusion headers for input/output streams, file handling, and vector/string containers.
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

// Source Attribution: Authored by student with assistance from Generative AI
// Description: Class definition for SudokuSolver encapsulating state and backtracking logic using objects.
class SudokuSolver {
private:
    // Source Attribution: Authored by student with assistance from Generative AI
    // Description: Private member variables storing the 9x9 board character array, filename, and solution status flag.
    char board[9][9];
    string fileName;
    bool foundAny;

    // Source Attribution: Authored by student with assistance from Generative AI
    // Description: Member function checking if a given digit is valid at a specified row and column.
    bool isValid(int row, int col, char digit) {
        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Loop iterating through all indices 0 to 8 to check row and column constraints.
        for (int i = 0; i < 9; i++) {
            // Source Attribution: Authored by student with assistance from Generative AI
            // Description: Returns false if the digit already exists in the current row.
            if (board[row][i] == digit) return false;
            // Source Attribution: Authored by student with assistance from Generative AI
            // Description: Returns false if the digit already exists in the current column.
            if (board[i][col] == digit) return false;
        }

        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Calculates the starting row index of the 3x3 sub-grid block.
        int startRow = row - (row % 3);
```

```
        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Calculates the starting column index of the 3x3 sub-grid block.
        int startCol = col - (col % 3);

        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Nested loop iterating through the 3x3 sub-grid block rows.
        for (int r = 0; r < 3; r++) {
            // Source Attribution: Authored by student with assistance from Generative AI
            // Description: Nested loop iterating through the 3x3 sub-grid block columns.
            for (int c = 0; c < 3; c++) {
                // Source Attribution: Authored by student with assistance from Generative AI
                // Description: Returns false if the digit is found anywhere within the 3x3 sub-grid
block.
                if (board[startRow + r][startCol + c] == digit) return false;
            }
        }

        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Returns true if no rule violations were detected for the chosen digit.
        return true;
    }

    // Source Attribution: Authored by student with assistance from Generative AI
    // Description: Recursive backtracking function to traverse cells and test candidates.
    void solve(int row, int col) {
        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Base case checking if row has reached 9, meaning a full valid board has
been completed.
        if (row == 9) {
            // Source Attribution: Authored by student with assistance from Generative AI
            // Description: Prints the completed board solution to standard output.
            printBoard();
            // Source Attribution: Authored by student with assistance from Generative AI
            // Description: Sets the flag indicating at least one valid solution has been found.
            foundAny = true;
            // Source Attribution: Authored by student with assistance from Generative AI
            // Description: Returns from the current recursive stack frame.
            return;
        }

        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Determines the row coordinate for the next recursive step, advancing if
column is 8.
        int nextRow = (col == 8) ? row + 1 : row;
```

```
        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Determines the column coordinate for the next recursive step, wrapping around to 0 if 8.
        int nextCol = (col == 8) ? 0 : col + 1;

        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Checks if the current cell is pre-filled with a character other than an underscore.
        if (board[row][col] != '_') {
            // Source Attribution: Authored by student with assistance from Generative AI
            // Description: Bypasses pre-filled cells by recursively calling solve on the next cell coordinates.
            solve(nextRow, nextCol);
        } else {
            // Source Attribution: Authored by student with assistance from Generative AI
            // Description: Iterates through character digits from '1' through '9' to test as candidates.
            for (char d = '1'; d <= '9'; d++) {
                // Source Attribution: Authored by student with assistance from Generative AI
                // Description: Tests if candidate digit 'd' is valid in the current empty cell.
                if (isValid(row, col, d)) {
                    // Source Attribution: Authored by student with assistance from Generative AI
                    // Description: Places candidate digit 'd' into the current cell board position.
                    board[row][col] = d;

                    // Source Attribution: Authored by student with assistance from Generative AI
                    // Description: Recursively calls solve for the next cell coordinates with the placed digit.
                    solve(nextRow, nextCol);

                    // Source Attribution: Authored by student with assistance from Generative AI
                    // Description: Backtracks by resetting the cell back to an underscore character for further searching.
                    board[row][col] = '_';
                }
            }
        }
    }

public:
    // Source Attribution: Authored by student with assistance from Generative AI
    // Description: Constructor initializing the solver object with a specific puzzle filename and default states.
    SudokuSolver(string name) {
```

```
        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Assigns the input filename string to the member variable.
        fileName = name;

        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Initializes foundAny tracking flag to false.
        foundAny = false;

        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Loop iterating across rows to initialize the 9x9 board array.
        for (int i = 0; i < 9; i++) {
            // Source Attribution: Authored by student with assistance from Generative AI
            // Description: Loop iterating across columns to clear board cells with underscores.
            for (int j = 0; j < 9; j++) {
                // Source Attribution: Authored by student with assistance from Generative AI
                // Description: Sets individual board cell to underscore character.
                board[i][j] = '_';
            }
        }
    }

    // Source Attribution: Authored by student with assistance from Generative AI
    // Description: Member function that opens and loads the puzzle file contents into the board
array.
    bool loadPuzzle() {
        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Declares and opens an input file stream using the stored filename.
        ifstream file(fileName);

        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Checks if the file stream failed to open and returns false if unsuccessful.
        if (!file.is_open()) {
            return false;
        }

        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Loop reading puzzle rows from the opened file stream.
        for (int i = 0; i < 9; i++) {
            // Source Attribution: Authored by student with assistance from Generative AI
            // Description: Loop reading puzzle columns and populating board grid elements.
            for (int j = 0; j < 9; j++) {
                // Source Attribution: Authored by student with assistance from Generative AI
                // Description: Extracts character tokens from file into the board matrix.
                file >> board[i][j];
```

```
            }
        }

        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Closes the input file stream after reading is complete.
        file.close();

        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Returns true indicating successful file loading.
        return true;
    }

    // Source Attribution: Authored by student with assistance from Generative AI
    // Description: Member function that prints the current contents of the Sudoku board.
    void printBoard() {
        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Loop iterating through each row of the board.
        for (int i = 0; i < 9; i++) {
            // Source Attribution: Authored by student with assistance from Generative AI
            // Description: Loop iterating through each column of the board row.
            for (int j = 0; j < 9; j++) {
                // Source Attribution: Authored by student with assistance from Generative AI
                // Description: Outputs board cell character followed by a space or empty string at row
end.
                cout << board[i][j] << (j == 8 ? "" : " ");
            }
            // Source Attribution: Authored by student with assistance from Generative AI
            // Description: Outputs a newline character at the end of each board row.
            cout << endl;
        }
    }

    // Source Attribution: Authored by student with assistance from Generative AI
    // Description: Driver member function to execute solving operations and print status headers
and results.
    void run() {
        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Prints the target puzzle file name indicator header.
        cout << "Puzzle File: " << fileName << endl;

        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Prints label for the original un-solved puzzle configuration.
        cout << "Original Puzzle:" << endl;
```

```
        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Calls printBoard to display the loaded initial puzzle state.
        printBoard();

        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Prints label header introducing solutions.
        cout << "\nSolution(s):" << endl;

        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Initiates the recursive backtracking search starting from top-left cell (0,0).
        solve(0, 0);

        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Checks if no solutions were found across the exhaustive search.
        if (!foundAny) {
            // Source Attribution: Authored by student with assistance from Generative AI
            // Description: Prints error message when no solution exists for the puzzle.
            cout << "No solution found" << endl;
        }

        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Prints a visual separator line between distinct puzzle output blocks.
        cout << "----------------------------------------" << endl;
    }
};

// Source Attribution: Authored by student with assistance from Generative AI
// Description: Main entry point function for executing the program logic across all puzzle files.
int main() {
    // Source Attribution: Authored by student with assistance from Generative AI
    // Description: Initializes an array containing the names of the five target puzzle text files.
    string files[5] = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.txt", "puzzle5.txt"};

    // Source Attribution: Authored by student with assistance from Generative AI
    // Description: Loop iterating sequentially over each of the five puzzle files.
    for (int i = 0; i < 5; i++) {
        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Instantiates a SudokuSolver object passing the current filename from the
array.
        SudokuSolver solver(files[i]);

        // Source Attribution: Authored by student with assistance from Generative AI
        // Description: Checks if the puzzle file successfully loads its contents.
        if (solver.loadPuzzle()) {
```

```
            // Source Attribution: Authored by student with assistance from Generative AI
            // Description: Executes the solver run routine for the loaded puzzle object.
            solver.run();
        } else {
            // Source Attribution: Authored by student with assistance from Generative AI
            // Description: Prints an error message if a file fails to open.
            cout << "Error: Could not open " << files[i] << endl;
        }
    }

    // Source Attribution: Authored by student with assistance from Generative AI
    // Description: Returns 0 to indicate successful program termination.
    return 0;
}
```

Claude.ai code (148 lines, excluding comments):

```
/*
 * PROLOG
 * Name:        <YOUR NAME>
 * Course/Lab:  <COURSE, ASSIGNMENT NAME>
 * Date:        October 15, 2026
 * Purpose:     Recursive, object-oriented, brute-force Sudoku solver that uses
 *              depth-first search with backtracking. It finds ALL solutions
 *              to each puzzle file and prints them.
 * Usage:       ./sudoku puzzle1.txt puzzle2.txt ...
 *              (with no arguments it solves puzzle1.txt through puzzle5.txt)
 *
 * Sources / authorship key (used in the "AUTHORSHIP" tag on each block):
 *   [GENAI-CLAUDE]  Code generated by Claude (Anthropic GenAI) from the
 *                   assignment text; I reviewed, compiled, and tested it.
 *   [ME]            Code I wrote myself.
 *   [COLLAB]        Code written by a collaborator.
 *   [MIX]           A combination of the above.
 *   Sources listed in prolog: Claude (Anthropic) -- generated the code.
 *   <ADD OTHER SOURCES, E.G. CHATGPT/COPILOT FOR YOUR SECOND GENAI, OR
NONE>
 */

#include <array>      // std::array: fixed-size container used for the 9x9 grid
#include <fstream>    // std::ifstream: reads the puzzle files
#include <iostream>   // std::cout / std::ostream: console output
#include <string>     // std::string: file names
#include <vector>     // std::vector: holds the list of solutions found

/*
 * AUTHORSHIP: [GENAI-CLAUDE]
 * Class Sudoku: one object represents one 9x9 puzzle grid. It knows how to
 * load itself from a file, print itself, check whether a digit may be placed
 * in a cell, and recursively search for every solution. All of the solving
 * logic lives in member functions of this object (no free functions).
 */
class Sudoku {
public:
    static const int N = 9;   // grid size: 9 rows and 9 columns

    /*
     * AUTHORSHIP: [GENAI-CLAUDE]
```

```
 * Constructor: remembers the file name, zeroes the grid (0 means blank),
 * then tries to load the puzzle from the file and records success/failure.
 */
explicit Sudoku(const std::string& filename) : name_(filename), loaded_(false) {
    for (auto& row : grid_) row.fill(0);   // set every cell in every row to 0 (blank)
    loaded_ = load(filename);              // read the file; store whether it worked
}

bool loaded() const { return loaded_; }          // true if the file held a valid 81-cell puzzle
const std::string& name() const { return name_; } // returns the puzzle's file name

/*
 * AUTHORSHIP: [GENAI-CLAUDE]
 * solveAll: runs the search and returns how many solutions exist.
 * It works on a COPY of the puzzle so the original givens stay intact
 * for printing, and stores every complete grid in solutions_.
 */
int solveAll() {
    solutions_.clear();                // discard results from any earlier call
    Sudoku work = *this;               // make a working copy of the puzzle
    work.search(solutions_);           // run the search; it appends each solution found
    return static_cast<int>(solutions_.size());  // number of solutions found
}

// Returns the list of solutions found by solveAll().
const std::vector<Sudoku>& solutions() const { return solutions_; }

/*
 * AUTHORSHIP: [GENAI-CLAUDE]
 * print: writes the grid to the given stream with separators between 3x3
 * blocks. Blank cells are shown as '_' and filled cells as their digit.
 */
void print(std::ostream& os) const {
    for (int r = 0; r < N; ++r) {                          // go through each row
        if (r % 3 == 0 && r != 0) os << "------+-------+------\n"; // horizontal line between block rows
        for (int c = 0; c < N; ++c) {                      // go through each column
            if (c % 3 == 0 && c != 0) os << "| ";          // vertical bar between block columns
            os << (grid_[r][c] ? char('0' + grid_[r][c]) : '_');   // digit if filled, '_' if blank
            os << (c < N - 1 ? " " : "");                  // space between cells (none after last)
        }
        os << '\n';                                        // end of the row
    }
}
```

```
private:
    std::string name_;                      // file name this puzzle came from
    bool loaded_;                           // did the file load correctly?
    std::array<std::array<int, N>, N> grid_;    // the 9x9 grid; 0 = blank, 1-9 = digit
    std::vector<Sudoku> solutions_;             // solutions found (used on the top-level object)

    /*
     * AUTHORSHIP: [GENAI-CLAUDE]
     * load: reads the file one character at a time. Digits 1-9 are givens;
     * '_' (also '0' or '.') is a blank. Whitespace and anything else is
     * skipped. Succeeds only if exactly 81 cells were read.
     */
    bool load(const std::string& filename) {
        std::ifstream in(filename);         // open the puzzle file
        if (!in) return false;              // file could not be opened
        int count = 0;                      // how many cells have been read so far
        char ch;                            // the character currently being examined
        while (in.get(ch) && count < N * N) {                   // read until EOF or 81 cells
            if (ch >= '1' && ch <= '9')                         // a given digit
                grid_[count / N][count % N] = ch - '0', ++count;   // store it at (row, col), advance
            else if (ch == '_' || ch == '0' || ch == '.') ++count; // a blank: leave 0, advance
        }
        return count == N * N;              // valid only if we got all 81 cells
    }

    /*
     * AUTHORSHIP: [GENAI-CLAUDE]
     * isCandidate: returns true if digit d can legally go in cell (r, c),
     * meaning d does not already appear in that row, column, or 3x3 block.
     */
    bool isCandidate(int r, int c, int d) const {
        for (int i = 0; i < N; ++i) {           // scan along the row and the column together
            if (grid_[r][i] == d) return false;  // d already in this row
            if (grid_[i][c] == d) return false;  // d already in this column
        }
        int br = (r / 3) * 3;                   // top row of the 3x3 block containing (r, c)
        int bc = (c / 3) * 3;                   // left column of that block
        for (int i = 0; i < 3; ++i)             // scan the 3 rows of the block
            for (int j = 0; j < 3; ++j)         // scan the 3 columns of the block
                if (grid_[br + i][bc + j] == d) return false;  // d already in this block
        return true;                            // no conflicts: d is a valid candidate
    }

    /*
```

```
 * AUTHORSHIP: [GENAI-CLAUDE]
 * findEmpty: finds the first blank cell in row-major order. Returns its
 * position through r and c; returns false if there are no blanks left.
 */
bool findEmpty(int& r, int& c) const {
    for (r = 0; r < N; ++r)                 // go through rows top to bottom
        for (c = 0; c < N; ++c)               // go through columns left to right
            if (grid_[r][c] == 0) return true;  // found a blank; r and c hold its position
    return false;                           // grid is full
}

/*
 * AUTHORSHIP: [GENAI-CLAUDE]
 * givensValid: checks that the digits already in the puzzle do not break
 * any rule (otherwise no solution can exist). Each given is temporarily
 * removed and tested as a candidate in its own cell, then put back.
 */
bool givensValid() {
    for (int r = 0; r < N; ++r)               // every row
        for (int c = 0; c < N; ++c) {         // every column
            int d = grid_[r][c];              // digit in this cell
            if (!d) continue;                 // skip blanks
            grid_[r][c] = 0;                  // temporarily remove the digit
            bool ok = isCandidate(r, c, d);  // would it be legal here with itself removed?
            grid_[r][c] = d;                  // restore the digit
            if (!ok) return false;            // a given conflicts with another given
        }
    return true;                            // all givens are consistent
}

/*
 * AUTHORSHIP: [GENAI-CLAUDE]
 * search: entry point for the search. Rejects puzzles whose givens already
 * conflict, otherwise starts the recursion.
 */
void search(std::vector<Sudoku>& found) {
    if (!givensValid()) return;   // contradictory puzzle: no solutions
    recurse(found);               // begin depth-first search
}

/*
 * AUTHORSHIP: [GENAI-CLAUDE]
 * recurse: the recursive depth-first search with backtracking.
 *   Base case: no blank cell remains (k == 81), so the grid is a complete
```

```
     *              valid solution; save a copy of it.
     *   Recursive case: pick the first blank cell, try each legal digit in it,
     *              recurse with one more cell filled, then undo the digit
     *              (backtrack) and try the next. It does not stop at the first
     *              solution, so every solution is found. If a cell has no
     *              candidates, the loop places nothing and the call simply
     *              returns, which reports "no solution" on this path.
     */
    void recurse(std::vector<Sudoku>& found) {
        int r, c;                           // position of the chosen blank cell
        if (!findEmpty(r, c)) {                 // base case: no blanks left
            Sudoku copy = *this;                // copy the completed grid
            copy.solutions_.clear();            // the copy does not need its own solution list
            found.push_back(copy);                 // record this solution
            return;                          // go back up to keep looking for more
        }
        for (int d = 1; d <= 9; ++d) {          // try every digit 1-9 in this cell
            if (isCandidate(r, c, d)) {         // only if it breaks no row/column/block rule
                grid_[r][c] = d;                // place the digit
                recurse(found);                    // recursive call with k+1 cells filled
                grid_[r][c] = 0;                // backtrack: remove the digit and try the next
            }
        }
    }
};

/*
 * AUTHORSHIP: [GENAI-CLAUDE]
 * main: builds the list of puzzle files (from the command line, or
 * puzzle1.txt..puzzle5.txt by default). For each file it creates a Sudoku
 * object, prints the file name and the puzzle, solves it, then prints each
 * solution or "No solution found".
 */
int main(int argc, char* argv[]) {
    std::vector<std::string> files;                     // names of the puzzle files to solve
    for (int i = 1; i < argc; ++i) files.push_back(argv[i]); // use any file names given on the
command line
    if (files.empty())                                  // none given: use the defaults
        for (int i = 1; i <= 5; ++i) files.push_back("puzzle" + std::to_string(i) + ".txt");

    for (const auto& f : files) {                       // process each puzzle file in turn
        std::cout << "==============================\n";     // separator line between
puzzles
        std::cout << "File: " << f << "\n\n";               // print the puzzle file name
```

```
        Sudoku puzzle(f);                               // create the puzzle object (loads the file)
        if (!puzzle.loaded()) {                         // file missing or not a valid 9x9 puzzle
            std::cout << "Could not read a valid 9x9 puzzle from this file.\n\n";
            continue;                                   // skip to the next file
        }

        std::cout << "Puzzle:\n";                       // label for the original puzzle
        puzzle.print(std::cout);                        // print the puzzle as read from the file
        std::cout << '\n';                              // blank line

        int n = puzzle.solveAll();                      // search; n = number of solutions found
        if (n == 0) {                                   // nothing found
            std::cout << "No solution found\n\n";       // required message for unsolvable puzzles
        } else {
            for (int i = 0; i < n; ++i) {               // print every solution
                std::cout << "Solution";                // heading for this solution
                if (n > 1) std::cout << " " << (i + 1) << " of " << n;  // number them if several
                std::cout << ":\n";                     // end of heading
                puzzle.solutions()[i].print(std::cout);  // print the solved grid
                std::cout << '\n';                      // blank line
            }
        }
    }
    return 0;                                           // normal exit
}
```

Final Code (126 lines, excluding comments)

```
/* Program Name: EECS 348 Assignment 4
* Brief Description: A recursive object-oriented brute-force
*                    sudoku puzzle solver
* Inputs: puzzle1.txt, puzzle2.txt, puzzle3.txt,
*         puzzle4.txt, puzzle5.txt
* Output: puzzle file name (i.e., puzzle1.txt), puzzle file contents,
*         and a solution to puzzle or "No solution found" printed out
* Collaborators: Claude.ai Sonnet 5.5 Medium to write hybrid code of sources' programs
* Other Sources: Claude.ai and Google Gemini
* Author's Full Name: Devan Mathis
* Creation Date: 10/15/2026
*/

#include <iostream>   // used for console output
#include <fstream>    // reads puzzle files
#include <vector>     // acts as a dynamic array to contain: list of solutions and file names
#include <string>     // file name type

using namespace std;

/*
* AUTHORSHIP:
*      Code: Claude.ai
*      Comments: Devan Mathis
*/
class SudokuSolver {
private:
  // represent puzzle w/ wrapper -> can be stored in a vector for printing
  // eliminates need for #include <array>, saving memory
  struct Grid { char cells[9][9]; }; // blank 9x9 grid--currently 0s
  char board[9][9];          // blank 9x9 char array
  string fileName;           // puzzle file name
  vector<Grid> solutions;    // grid of solutions

  /*
  * AUTHORSHIP:
  *      Code: Claude.ai
  *      Comments: Devan Mathis
  */
  // checks if a digit is already present in (row, col) of 3x3 section of puzzle:
  // present --> false, not present --> true
```

```
bool isValid(int row, int col, char digit) const {
   for (int i = 0; i < 9; i++) {
      if (board[row][i] == digit) return false;   // already in row
      if (board[i][col] == digit) return false;   // already in column
   }
   int startRow = row - (row % 3);                 // adjust to top of 3x3 block
   int startCol = col - (col % 3);                 // adjust to left of 3x3 block
   for (int r = 0; r < 3; r++)
      for (int c = 0; c < 3; c++)
         if (board[startRow + r][startCol + c] == digit) return false;
   return true;                                    // not present
}

/*
* AUTHORSHIP:
*      Code: Claude.ai
*      Comments: Devan Mathis
*/
// recursive backtracking over cells' digits, one cell per run
void solve(int row, int col) {
   if (row == 9) {                                 // base case: 9x9
      Grid g;                                      // row == 9, thus full grid
      for (int i = 0; i < 9; i++)
         for (int j = 0; j < 9; j++)
            g.cells[i][j] = board[i][j];           // copy solution
      solutions.push_back(g);                      // record in g grid
      return;
   }
   int nextRow = (col == 8) ? row + 1 : row;       // next cell's row
   int nextCol = (col == 8) ? 0 : col + 1;         // next cell's column
   if (board[row][col] != '_') {                   // pre-filled cell
      solve(nextRow, nextCol);
   } else {
      for (char d = '1'; d <= '9'; d++) {          // try each digit
         if (isValid(row, col, d)) {
            board[row][col] = d;                   // place
            solve(nextRow, nextCol);               // recurse
            board[row][col] = '_';                 // backtrack
         }
      }
   }
}
/*
* AUTHORSHIP:
```

```
    *      Code: Claude.ai
    *      Comments: Devan Mathis
    */
    // prints a given 9x9 grid w/ seprators between all 3x3 blocks
    void printGrid(const char g[9][9]) const {
        for (int r = 0; r < 9; r++) {
            if (r % 3 == 0 && r != 0) cout << "------+-------+------\n";
            for (int c = 0; c < 9; c++) {
                if (c % 3 == 0 && c != 0) cout << "| ";   // block divider
                cout << g[r][c] << (c < 8 ? " " : "");    // cell + spacing
            }
            cout << '\n';
        }
    }
public:
    /*
    * AUTHORSHIP:
    *      Code: Claude.ai
    *      Comments: Devan Mathis
    */
    // blanks a board and stores a corresponding filename w/ it
    SudokuSolver(string name) {
        fileName = name;
        for (int i = 0; i < 9; i++)
            for (int j = 0; j < 9; j++)
                board[i][j] = '_';
    }
    /*
    * AUTHORSHIP:
    *      Code: Claude.ai
    *      Comments: Devan Mathis
    */
    // Checks if puzzle file can load a 9x9 grid: yes-->true, no-->false
    bool loadPuzzle() {
        ifstream file(fileName);
        if (!file.is_open()) return false; // can't be opened/DNE
        for (int i = 0; i < 9; i++)
            for (int j = 0; j < 9; j++)
                if (!(file >> board[i][j])) return false;   // ran out of data
        file.close();
        return true;
    }
    /*
    * AUTHORSHIP:
```

```
  *      Code: Claude.ai
  *      Comments: Devan Mathis
  */
  // method to run program; acts as a main file, while main checks for conflicts
  void run() {
     cout << "Puzzle:\n";
     printGrid(board);                                  // original puzzle
     cout << '\n';
     solve(0, 0);                                       // find all solutions
     int n = static_cast<int>(solutions.size());
     if (n == 0) {
        cout << "No solution found\n\n";                 // no solution found
     } else {
        for (int i = 0; i < n; i++) {
           cout << "Solution";
           if (n > 1) cout << " " << (i + 1) << " of " << n; // print solution count
           cout << ":\n";
           printGrid(solutions[i].cells);               // print solution grids
           cout << '\n';
        }
     }
  }
};
/*
* AUTHORSHIP:
*      Code: Claude.ai
*      Comments: Devan Mathis
*/
// reads input file(s), then mainly checks for errors before running whole program
int main(int argc, char* argv[]) {
  vector<string> files;
  for (int i = 1; i < argc; ++i) files.push_back(argv[i]);   // command-line files
  if (files.empty())                                    // none given: defaults
     for (int i = 1; i <= 5; ++i) files.push_back("puzzle" + to_string(i) + ".txt");
  for (size_t i = 0; i < files.size(); ++i) {
     cout << "==============================\n";          // formatting separator
     cout << "File: " << files[i] << "\n\n";             // file name
     SudokuSolver solver(files[i]);                     // load solver object w/ puzzle files
     if (solver.loadPuzzle()) {
        solver.run();                                   // run whole program from solver object
     } else {
        cout << "Could not read a valid 9x9 puzzle from this file.\n\n";
     }
  }
```

```
    return 0;
}
```